

Trabajo Practico 1: Mapas perceptuales y Análisis Factorial

Maestría en Generación y Análisis de Información Estadística - UNTREF

Andrea Gomez Vargas & Karen Ulloa

24/08/2026

Tabla de contenidos

Caso de estudio: lealtad de consumidores	2
Lealtad cognitiva	2
Lealtad afectiva	2
Objetivo del análisis	3
Carga de paquetes y datos	3
Exploración inicial	3
Selección de variables	5
Test de esfericidad de Bartlett	5
Medida de adecuación muestral KMO	5
Matriz de correlaciones	6
Autovalores	8
Gráfico de sedimentación	9
Análisis factorial con dos factores	10
Matriz de cargas factoriales rotadas	11
Comunalidades	12
Varianza explicada	12

Caso de estudio: lealtad de consumidores

El objetivo de este análisis es identificar la estructura subyacente en un conjunto de ítems utilizados para medir la **lealtad de los consumidores hacia una marca de zapatillas**.

La empresa *Shoetas* desea evaluar el grado de lealtad de sus consumidores. Para ello, se entrevistó mediante un cuestionario a **189 personas** en una tienda de artículos deportivos.

El estudio considera inicialmente dos dimensiones de la lealtad: **lealtad cognitiva** y **lealtad afectiva**. Todos los ítems fueron medidos mediante una escala Likert de cinco puntos, donde:

- **1 = completamente en desacuerdo**
- **5 = completamente de acuerdo**

Lealtad cognitiva

La lealtad cognitiva fue evaluada mediante seis afirmaciones:

- **CLOY1:** Es preferible utilizar las zapatillas Shoetas.
- **CLOY2:** Las características generales de las zapatillas Shoetas corresponden con mis expectativas.
- **CLOY3:** Si alguien me propusiera utilizar otra marca de zapatillas, continuaría utilizando Shoetas por su diseño.
- **CLOY4:** Si alguien me propusiera utilizar otra marca de zapatillas, continuaría utilizando Shoetas por su durabilidad.
- **CLOY5:** Si alguien me propusiera utilizar otra marca de zapatillas, continuaría utilizando Shoetas por su calidad.
- **CLOY6:** Si alguien me propusiera utilizar otra marca de zapatillas, continuaría utilizando Shoetas por su precio.

Lealtad afectiva

La lealtad afectiva fue evaluada mediante tres afirmaciones:

- **ALOY1:** Me gustan las zapatillas Shoetas más que otras zapatillas.
- **ALOY2:** Me gustan las características de mis zapatillas Shoetas.
- **ALOY3:** Tengo una actitud positiva hacia las zapatillas Shoetas.

Objetivo del análisis

A partir de estos nueve ítems se realizará un **análisis factorial exploratorio** con el objetivo de determinar si las variables pueden resumirse en un número menor de factores subyacentes y evaluar si la estructura obtenida coincide con las dimensiones teóricas de **lealtad cognitiva** y **lealtad afectiva**.

Carga de paquetes y datos

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from factor_analyzer import FactorAnalyzer
from factor_analyzer.factor_analyzer import (
    calculate_bartlett_sphericity,
    calculate_kmo
)
```

```
# Ruta del archivo
Loyalty = "data/Loyalty.xls"

# Leer archivo Excel
df = pd.read_excel(Loyalty)
```

Exploración inicial

```
# Ver las primeras filas
print("Primeras filas de la base:")
display(df.head())

# Dimensiones
print("\nDimensiones de la base:")
print(df.shape)

# Nombres de las variables
print("\nVariables:")
print(df.columns.tolist())
```

```
# Información general
print("\nInformación de la base:")
df.info()
```

Primeras filas de la base:

	ALOY1	ALOY2	ALOY3	CLOY1	CLOY2	CLOY3	CLOY4	CLOY5	CLOY6	id
0	5	5	5	5	5	5	5	5	5	1
1	4	5	3	5	5	5	4	4	4	2
2	3	4	3	4	4	4	4	4	3	3
3	4	4	4	4	4	4	4	4	4	4
4	3	3	3	2	3	2	2	2	3	5

Dimensiones de la base:
(189, 10)

Variables:

```
['ALOY1', 'ALOY2', 'ALOY3', 'CLOY1', 'CLOY2', 'CLOY3', 'CLOY4', 'CLOY5', 'CLOY6', 'id']
```

Información de la base:

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 189 entries, 0 to 188
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	ALOY1	189 non-null	int64
1	ALOY2	189 non-null	int64
2	ALOY3	189 non-null	int64
3	CLOY1	189 non-null	int64
4	CLOY2	189 non-null	int64
5	CLOY3	189 non-null	int64
6	CLOY4	189 non-null	int64
7	CLOY5	189 non-null	int64
8	CLOY6	189 non-null	int64
9	id	189 non-null	int64

```
dtypes: int64(10)
```

```
memory usage: 14.9 KB
```

Selección de variables

```
variables = [  
    "ALOY1", "ALOY2", "ALOY3",  
    "CLOY1", "CLOY2", "CLOY3",  
    "CLOY4", "CLOY5", "CLOY6"  
]  
  
datos = df[variables].copy()
```

Test de esfericidad de Bartlett

```
chi_square, p_value = calculate_bartlett_sphericity(datos)  
  
print("TEST DE ESFERICIDAD DE BARTLETT")  
print("-----")  
print("Chi-cuadrado:", round(chi_square, 3))  
print("p-value:", p_value)
```

```
TEST DE ESFERICIDAD DE BARTLETT  
-----  
Chi-cuadrado: 1458.09  
p-value: 3.2038978831038176e-283
```

Medida de adecuación muestral KMO

```
kmo_variables, kmo_total = calculate_kmo(datos)  
  
print("KMO")  
print("-----")  
print("KMO global:", round(kmo_total, 3))  
  
print("\nKMO por variable:")  
  
kmo_df = pd.DataFrame({  
    "Variable": variables,  
    "KMO": kmo_variables
```

```
} )
```

```
display(kmo_df.round(3))
```

KMO

KMO global: 0.865

KMO por variable:

	Variable	KMO
0	ALOY1	0.854
1	ALOY2	0.825
2	ALOY3	0.864
3	CLOY1	0.916
4	CLOY2	0.877
5	CLOY3	0.853
6	CLOY4	0.886
7	CLOY5	0.857
8	CLOY6	0.857

Matriz de correlaciones

```
matriz_correlacion = datos.corr()  
  
print("MATRIZ DE CORRELACIONES")  
display(matriz_correlacion.round(3))
```

MATRIZ DE CORRELACIONES

	ALOY1	ALOY2	ALOY3	CLOY1	CLOY2	CLOY3	CLOY4	CLOY5	CLOY6
ALOY1	1.000	0.855	0.652	0.422	0.464	0.456	0.471	0.478	0.694
ALOY2	0.855	1.000	0.695	0.385	0.478	0.487	0.451	0.455	0.756
ALOY3	0.652	0.695	1.000	0.330	0.402	0.368	0.350	0.338	0.792
CLOY1	0.422	0.385	0.330	1.000	0.689	0.663	0.723	0.748	0.448
CLOY2	0.464	0.478	0.402	0.689	1.000	0.831	0.718	0.694	0.463

	ALOY1	ALOY2	ALOY3	CLOY1	CLOY2	CLOY3	CLOY4	CLOY5	CLOY6
CLOY3	0.456	0.487	0.368	0.663	0.831	1.000	0.712	0.635	0.442
CLOY4	0.471	0.451	0.350	0.723	0.718	0.712	1.000	0.827	0.445
CLOY5	0.478	0.455	0.338	0.748	0.694	0.635	0.827	1.000	0.426
CLOY6	0.694	0.756	0.792	0.448	0.463	0.442	0.445	0.426	1.000

```

fig, ax = plt.subplots(figsize=(9, 7))

im = ax.imshow(
    matriz_correlacion,
    vmin=-1,
    vmax=1,
    cmap="PuOr"
)

ax.set_xticks(range(len(matriz_correlacion.columns)))
ax.set_yticks(range(len(matriz_correlacion.columns)))

ax.set_xticklabels(
    matriz_correlacion.columns,
    rotation=45,
    ha="right"
)

ax.set_yticklabels(matriz_correlacion.columns)

for i in range(len(matriz_correlacion.columns)):
    for j in range(len(matriz_correlacion.columns)):
        ax.text(
            j,
            i,
            f"{matriz_correlacion.iloc[i, j]:.2f}",
            ha="center",
            va="center"
        )

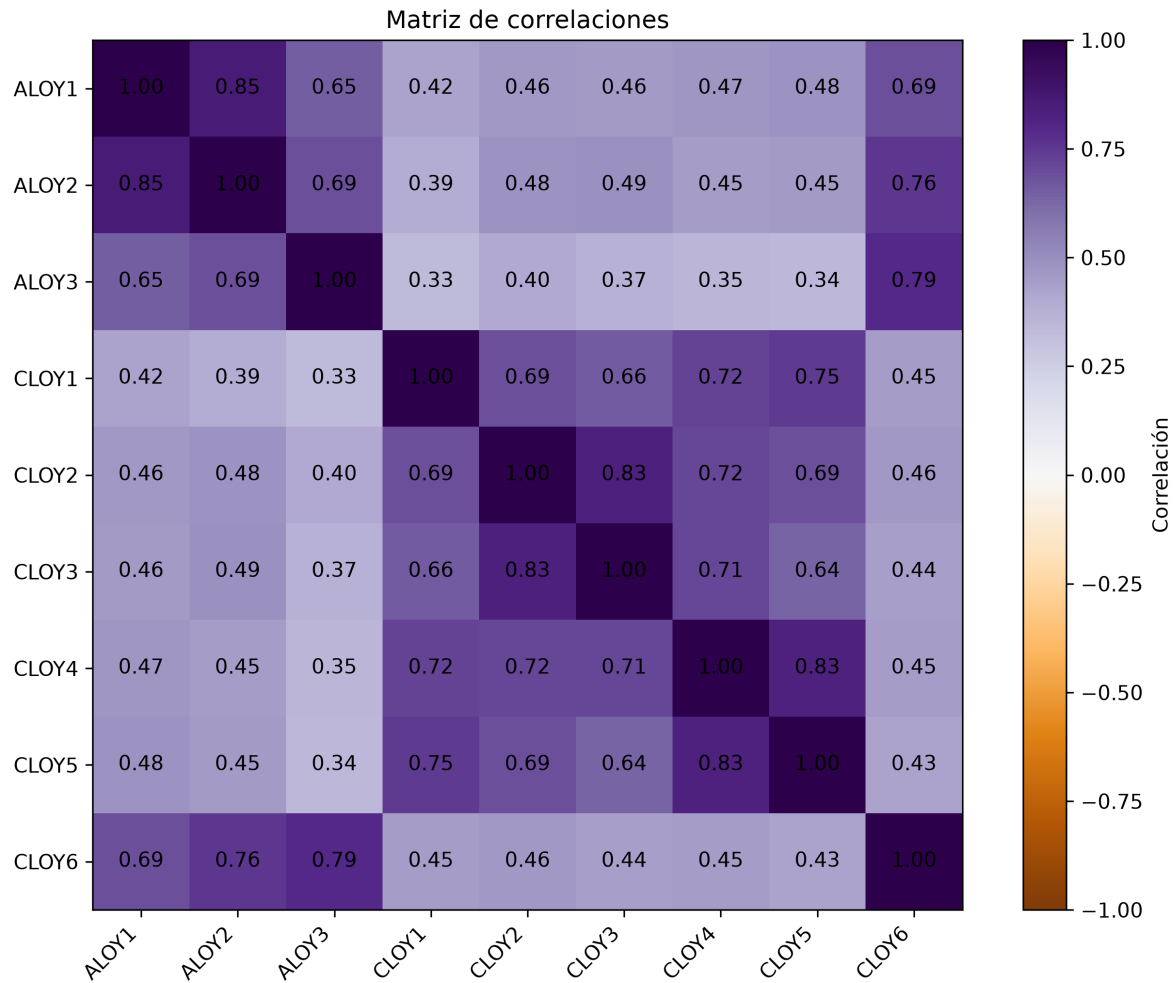
fig.colorbar(im, ax=ax, label="Correlación")

ax.set_title("Matriz de correlaciones")

plt.tight_layout()

```

```
plt.show()
```



Autovalores

```
fa_inicial = FactorAnalyzer(  
    n_factors=9,  
    rotation=None,  
    method="principal"  
)  
  
fa_inicial.fit(datos)
```



```
eigenvalues, _ = fa_inicial.get_eigenvalues()

tabla_autovalores = pd.DataFrame({
    "Factor": range(1, len(eigenvalues) + 1),
    "Autovalor": eigenvalues
})

display(tabla_autovalores.round(3))
```

```
C:\Users\andre\Documents\GitHub\estadistica-aplicada\.venv\Lib\site-packages\sklearn\utils\deprecation.py:151: DeprecationWarning:
warnings.warn(
```

	Factor	Autovalor
0	1	5.510
1	2	1.624
2	3	0.482
3	4	0.440
4	5	0.289
5	6	0.206
6	7	0.180
7	8	0.147
8	9	0.122

Gráfico de sedimentación

```
plt.figure(figsize=(8, 5))

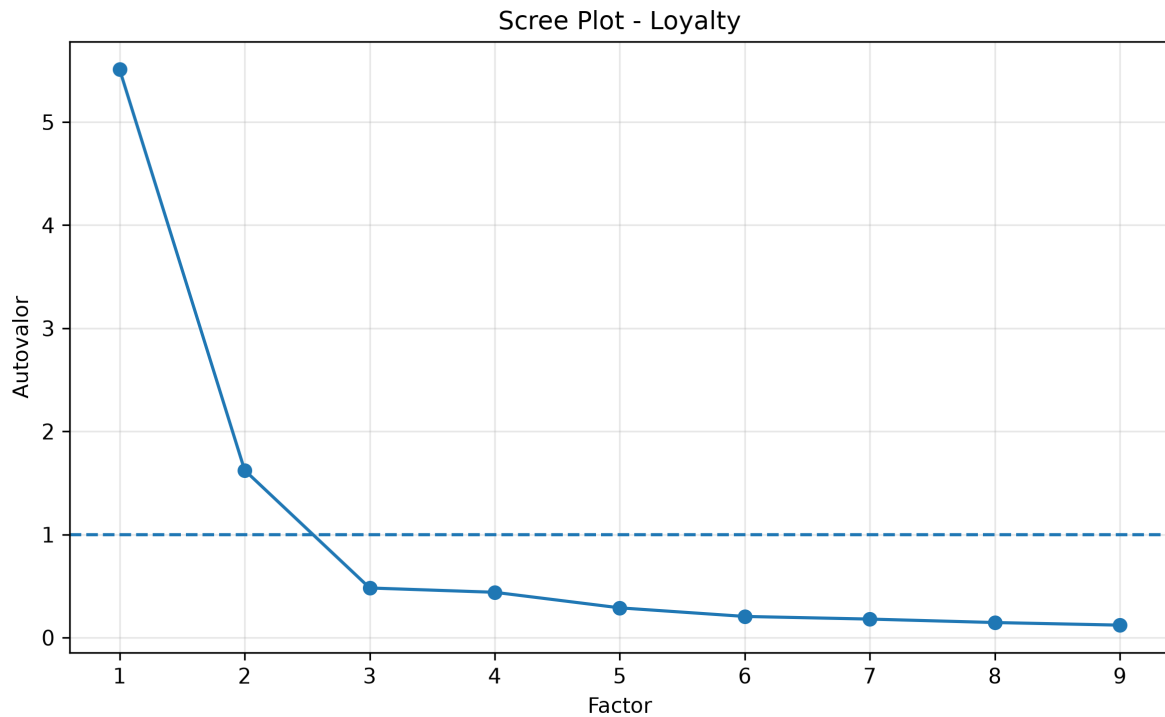
plt.plot(
    range(1, len(eigenvalues) + 1),
    eigenvalues,
    marker="o"
)

plt.axhline(y=1, linestyle="--")

plt.xlabel("Factor")
plt.ylabel("Autovalor")
plt.title("Scree Plot - Loyalty")
```

```
plt.xticks(range(1, len(eigenvalues) + 1))
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



A partir del gráfico de sedimentación se observa aproximadamente:

- Factor 1: autovalor 5,5
- Factor 2: autovalor 1,6
- Factor 3 en adelante: autovalores menores que 1.

Análisis factorial con dos factores

```
fa = FactorAnalyzer(
    n_factors=2,
    rotation="varimax",
    method="principal"
)
```

```
fa.fit(datos)
```

```
C:\Users\andre\Documents\GitHub\estadistica-aplicada\.venv\Lib\site-packages\sklearn\utils\deprecation.py:101: DeprecationWarning:
warnings.warn(
```

n_factors	2
rotation	'varimax'
method	'principal'
use_smc	True
is_corr_matrix	False
bounds	(0.005, ...)
impute	'median'
svd_method	'randomized'
rotation_kwargs	{}

Matriz de cargas factoriales rotadas

```
cargas = pd.DataFrame(
    fa.loadings_,
    index=variables,
    columns=["Factor 1", "Factor 2"]
)

print("MATRIZ DE CARGAS FACTORIALES ROTADAS")
display(cargas.round(3))
```

MATRIZ DE CARGAS FACTORIALES ROTADAS

	Factor 1	Factor 2
ALOY1	0.314	0.831
ALOY2	0.291	0.873
ALOY3	0.169	0.868
CLOY1	0.845	0.197
CLOY2	0.845	0.273
CLOY3	0.826	0.263
CLOY4	0.875	0.229

	Factor 1	Factor 2
CLOY5	0.859	0.226
CLOY6	0.282	0.858

Comunalidades

```
comunalidades = pd.DataFrame({
    "Variable": variables,
    "Comunalidad": fa.get_communalities()
})

print("COMUNALIDADES")
display(comunalidades.round(3))
```

COMUNALIDADES

	Variable	Comunalidad
0	ALOY1	0.789
1	ALOY2	0.847
2	ALOY3	0.782
3	CLOY1	0.753
4	CLOY2	0.788
5	CLOY3	0.751
6	CLOY4	0.818
7	CLOY5	0.789
8	CLOY6	0.817

Varianza explicada

```
varianza = fa.get_factor_variance()

tabla_varianza = pd.DataFrame({
    "Factor": ["Factor 1", "Factor 2"],
    "SS Loadings": varianza[0],
    "Proporción de varianza": varianza[1],
    "Varianza acumulada": varianza[2]
})
```

```
}  
  
print("VARIANZA EXPLICADA")  
display(tabla_varianza.round(3))
```

VARIANZA EXPLICADA

	Factor	SS Loadings	Proporción de varianza	Varianza acumulada
0	Factor 1	3.905	0.434	0.434
1	Factor 2	3.229	0.359	0.793